

Talento Altamente Especializado – Inteligencia Artificial 2026 COCYTEN-Nayarit

Lab: Implement and evaluate a Neural Network for segmentation and object detection using COCO

Student: Stephany Anahí López Lizárraga

Instructor: M.Sc. Mario Ivan Lopez Valdovinos

Assignment Date: 24/07/2026





Cinvestav
Guadalajara



COCYTEN
CONSEJO DE CIENCIA Y TECNOLOGÍA
DEL ESTADO DE NAYARIT



Nayarit
NUESTRA DIGNIDAD Y COMPROMISO



INTRODUCTION

This project focuses on implementing and evaluating neural network models for object detection and instance segmentation using the COCO dataset. The models are trained to identify multiple object categories, locate them using bounding boxes, and generate pixel-level segmentation masks. Their performance is assessed using standard metrics such as precision, recall, Intersection over Union (IoU), and mean Average Precision (mAP).



1. ESTRUCTURA DEL LABORATORIO

1.1. INSTALACIÓN DE DEPENDENCIAS

Antes de ejecutar el notebook es necesario contar con Python, PyTorch, Ultralytics (YOLO), Matplotlib, pycocotools (para leer las anotaciones oficiales de COCO en formato JSON) y OpenCV. Todo se instala con un solo comando desde el propio notebook:

Se instalan con un solo comando desde la raíz del repositorio:

```
%pip install -q ultralytics matplotlib pycocotools pyyaml opencv-python
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell - nayarit_base_cnn + x

(modulo4) (base) PS C:\Users\User\Desktop\Módulo 4\Labs\Proyecto Final\Nayarit_base_cnn> %pip install -q ultralytics matplotlib pycocotools pyyaml opencv-python
Collecting ultralytics
  Downloading ultralytics-8.2.0-py3-none-any.whl (750 kB)
    750/750 kB 9.8 MB/s eta 0:00:00
Collecting matplotlib
  Downloading matplotlib-3.9.0-cp311-cp311-win_amd64.whl (7.8 MB)
    7.8/7.8 MB 12.1 MB/s eta 0:00:00
Installing collected packages: pyyaml, pycocotools, opencv-python, matplotlib, ultralytics
Successfully installed matplotlib-3.9.0 opencv-python-4.10.0 pycocotools-2.0.7 pyyaml-6.0.1 ultralytics-8.2.0

(modulo4) (base) PS C:\Users\User\Desktop\Módulo 4\Labs\Proyecto Final\Nayarit_base_cnn>
```

El siguiente bloque es la celda de configuración central del proyecto, define de una sola vez todos los valores que vas a reutilizar más adelante en el notebook. Se divide en varias partes:

1. Reproducibilidad

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```



2. Rutas

```
# --- Rutas ---
COCO_RAW_ROOT = Path("./coco_raw").resolve() # se descargan anotaciones + imágenes originales de COCO
DATASET_ROOT = Path("./coco_lab_dataset").resolve() # se arma nuestro propio split 70/15/15
```

3. Proporciones del split

```
# --- Subconjunto y split ---
# El enunciado permite usar un subconjunto en vez de COCO completo (123,287 imgs)
SUBSET_SIZE = 2500 # cuántas imágenes de val2017 se usan en total
TRAIN_FRACTION = 0.70 # % para entrenamiento
VAL_FRACTION = 0.15 # % para validación (monitoreo durante el entrenamiento)
TEST_FRACTION = 0.15 # % para prueba final (el resto, no se declara explícito)
```

4. Hiperparámetros de entrenamiento

```
# --- Entrenamiento ---
DET_WEIGHTS = "yolo11n.pt" # pesos preentrenados de detección (punto de partida, no entrena desde cero)
SEG_WEIGHTS = "yolo11n-seg.pt" # pesos preentrenados de segmentación (mismo caso)
EPOCHS = 5 # pocas épocas alcanzan porque ya viene preentrenado en estas mismas clases
IMG_SIZE = 416 # tamaño de imagen de entrada -> balance velocidad/calidad
BATCH_SIZE = 16 # imágenes procesadas por paso -> bajar si falta RAM
PATIENCE = 3 # corta el entrenamiento si no mejora en 3 épocas seguidas
CACHE_MODE = "disk" # cachea imágenes ya procesadas para no releer/decodificar JPEGs cada época
WORKERS = max(1, (os.cpu_count() or 4) - 1) # hilos de carga de datos según núcleos disponibles
DEVICE = 0 if torch.cuda.is_available() else "cpu" # usa GPU si hay, si no CPU
```

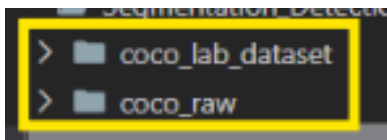
5. Visualización

```
# --- Visualización ---
N_VIS_EXAMPLES = 6 # cuántos ejemplos de test se muestran al final (Sección 12)
```

6. Crear carpetas y confirmar

```
DATASET_ROOT.mkdir(parents=True, exist_ok=True) # crea la carpeta del dataset si no existe
COCO_RAW_ROOT.mkdir(parents=True, exist_ok=True) # crea la carpeta de descarga si no existe
print(f"COCO_RAW_ROOT = {COCO_RAW_ROOT}")
print(f"DATASET_ROOT = {DATASET_ROOT}")
print(f"CPU detectado: {os.cpu_count()} núcleos -> WORKERS = {WORKERS}")
```

7. Resultado de creación de carpetas



```
COCO_RAW_ROOT = E:\28\3. Taller Altamente Especializado IA - CINVESTAV\Módulo 4\Laboratorios\Segmentation_Detection\coco_raw
DATASET_ROOT = E:\28\3. Taller Altamente Especializado IA - CINVESTAV\Módulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset
CPU detectado: 6 núcleos -> WORKERS = 5
```



1.2. DESCARGAR DEL DATASET COCO (SUBCONJUNTO)

Primero se descargan solo las anotaciones oficiales (250 MB). Las imágenes NO se descargan como un .zip completo (val2017.zip tiene 5,000 imágenes, 1 GB), más adelante se descarga, imagen por imagen, únicamente el subconjunto que realmente se va a usar.

```
# Solo descargamos anotaciones (250 MB); las imágenes se bajan después, una por una
COCO_FILES = {
    "annotations_trainval2017.zip": "http://images.cocodataset.org/annotations/annotations_trainval2017.zip",
}

def download_with_progress(url, dest):
    def _hook(block_num, block_size, total_size): # se llama automáticamente en cada bloque descargado
        downloaded = block_num * block_size
        if total_size > 0:
            pct = min(100, downloaded * 100 / total_size)
            print(f"\r {dest.name}: {pct:5.1f}%", end="") # imprime % de avance en la misma línea
    urllib.request.urlretrieve(url, dest, reporthook=_hook) # descarga el archivo mostrando progreso
    print()

for fname, url in COCO_FILES.items():
    dest = COCO_RAW_ROOT / fname
    if dest.exists(): # si ya está descargado, no lo vuelve a bajar
        print(f"{fname} ya descargado, se omite.")
        continue
    print(f"Descargando {fname} ...")
    download_with_progress(url, dest)

for fname in COCO_FILES:
    marker = COCO_RAW_ROOT / (fname + ".extracted") # archivo "bandera" para saber si ya se descomprimió
    if marker.exists():
        print(f"{fname} ya descomprimido, se omite.")
        continue
    print(f"Descomprimiendo {fname} ...")
    with zipfile.ZipFile(COCO_RAW_ROOT / fname) as zf:
        zf.extractall(COCO_RAW_ROOT) # descomprime el .zip de anotaciones
    marker.touch() # marca como ya descomprimido

print("Anotaciones descargadas y extraídas.")
print("Contenido de COCO_RAW_ROOT:", [p.name for p in COCO_RAW_ROOT.iterdir()]) # confirma qué quedó en la carpeta
```

A continuación, se muestra la salida de la descarga/extracción de las anotaciones:

```
Descargando annotations_trainval2017.zip ...
  annotations_trainval2017.zip: 100.0%
Descomprimiendo annotations_trainval2017.zip ...
Anotaciones descargadas y extraídas.
Contenido de COCO_RAW_ROOT: ['annotations', 'annotations_trainval2017.zip', 'annotations_trainval2017.zip.extracted']
```



1.3. PARTICIÓN DEL DATASET: 70% TRAIN / 15% VAL / 15% TEST

De las 5,000 imágenes anotadas de val2017, se toma aleatoriamente un subconjunto de 2,500 (semilla fija para reproducibilidad), tal como permite el enunciado. Sobre esas 2,500 imágenes se aplica una partición aleatoria propia en proporción 70/15/15, a nivel de imagen completa, para que todas las anotaciones de una imagen queden siempre en el mismo split.

Tamaño de cada partición obtenida:

- Entrenamiento (70%): 1,750 imágenes
- Validación (15%): 375 imágenes
- Prueba (15%): 375 imágenes

El siguiente código carga las anotaciones de val2017, toma aleatoriamente un subconjunto de 2,500 imágenes, y lo divide en las tres particiones propias de forma reproducible, sin perder ni duplicar imágenes por redondeo.

```
ann_val_path = COCO_RAW_ROOT / "annotations" / "instances_val2017.json" # ruta al JSON de anotaciones de val2017

print("Cargando anotaciones (puede tardar un momento)...")
coco_val = COCO(str(ann_val_path)) # carga el archivo de anotaciones con pycocotools

# lista completa de imágenes anotadas disponibles en val2017 (5,000 imágenes)
all_images = []
for image_id, image_info in coco_val.imgs.items(): # recorre todas las imágenes del JSON
    all_images.append({
        "coco_api": coco_val, # referencia a la API para consultar anotaciones después
        "subdir": "val2017", # carpeta original donde vivirá la imagen descargada
        "image_id": image_id, # ID único de la imagen en COCO
        "file_name": image_info["file_name"], # nombre real del archivo .jpg
        "width": image_info["width"], # ancho original en píxeles
        "height": image_info["height"], # alto original en píxeles
    })

random.shuffle(all_images) # mezcla aleatoriamente antes de recortar el subconjunto

# Tomamos solo SUBSET_SIZE imágenes (Sección 0) para que el entrenamiento sea manejable en CPU
subset_images = all_images[:min(SUBSET_SIZE, len(all_images))] # recorta al tamaño definido

n_total = len(subset_images)
n_train = int(TRAIN_FRACTION * n_total) # cuántas imágenes van a entrenamiento
n_val = int(VAL_FRACTION * n_total) # cuántas van a validación
# El resto va a test, para no perder/duplicar imágenes por redondeo
n_test = n_total - n_train - n_val

split_entries = {
    "train": subset_images[:n_train], # primer bloque -> entrenamiento
    "val": subset_images[n_train:n_train + n_val], # segundo bloque -> validación
    "test": subset_images[n_train + n_val:], # resto -> prueba
}

print(f"Imágenes anotadas disponibles en val2017: {len(all_images)}")
print(f"Subconjunto usado para este laboratorio: {n_total}")
for split_name, entries in split_entries.items():
    print(f" {split_name:5s}: {len(entries):7d} imágenes ({100*len(entries)/n_total:.1f}%) # reporte final del split
```



A continuación, se muestra la salida con el conteo real de imágenes por split (train/val/test):

```
Cargando anotaciones (puede tardar un momento)...
loading annotations into memory...
Done (t=0.55s)
creating index...
index created!
Imágenes anotadas disponibles en val2017: 5000
Subconjunto usado para este laboratorio: 2500
train: 1750 imágenes (70.0%)
val : 375 imágenes (15.0%)
test : 375 imágenes (15.0%)
```

Descarga individual de únicamente las imágenes seleccionadas:

```
val2017_dir = COCO_RAW_ROOT / "val2017" # carpeta donde se van a guardar las imágenes descargadas
val2017_dir.mkdir(parents=True, exist_ok=True)

# Unimos las entradas de los 3 splits -- son las únicas imágenes que de verdad necesitamos.
images_needed = [entry for entries in split_entries.values() for entry in entries] # aplana train+val+test en una sola lista

downloaded, skipped, failed = 0, 0, 0 # contadores para el resumen final
for idx, entry in enumerate(images_needed, start=1):
    dest = val2017_dir / entry["file_name"]
    if dest.exists(): # si ya se descargó antes, no la vuelve a bajar (permite reanudar)
        skipped += 1
        continue
    image_info = entry["coco_api"].imgs[entry["image_id"]]
    url = image_info.get("coco_url") or f"http://images.cocodataset.org/val2017/{entry['file_name']}" # URL directa de esa imagen
    try:
        urllib.request.urlretrieve(url, dest) # descarga la imagen individual
        downloaded += 1
    except Exception as e:
        failed += 1 # guarda el fallo pero no detiene el proceso
        print(f" Falló {entry['file_name']}: {e}")

    if idx % 100 == 0 or idx == len(images_needed): # imprime avance cada 100 imágenes
        print(f"\rProgreso: {idx}/{len(images_needed)} (nuevas: {downloaded}, ya existían: {skipped}, fallidas: {failed})", end="")

print()
print(f"Descarga del subconjunto completa: {downloaded} nuevas, {skipped} ya existían, {failed} fallidas.")
if failed:
    print("Vuelve a correr esta misma celda para reintentar las que fallaron (por timeouts, etc.).") # reanudable ante fallos de red
```

A continuación, se muestra la salida del progreso de descarga de las 2,500 imágenes individuales:

```
Progreso: 2500/2500 (nuevas: 2500, ya existían: 0, fallidas: 0)
Descarga del subconjunto completa: 2500 nuevas, 0 ya existían, 0 fallidas.
```



1.4. PREPARACIÓN DE LAS ANOTACIONES

Se remapean las 80 categorías de COCO a índices contiguos 0-79, y se define una función que convierte cada anotación (bbox + polígono) a las dos líneas de formato YOLO que se necesitan.

```
cat_ids = sorted(coco_val.getCatIds()) # IDs de categoría de COCO (van de 1 a 90, con huecos)
catid_to_yoloidx = {cat_id: idx for idx, cat_id in enumerate(cat_ids)} # remapea a índices contiguos 0-79 para YOLO
class_names = [coco_val.loadCats(cat_id)[0]["name"] for cat_id in cat_ids] # nombre legible de cada clase, en el mismo orden
NUM_CLASSES = len(class_names)
print(f"Número de clases COCO: {NUM_CLASSES}")
print("Primeras 5 clases:", class_names[:5]) # solo para verificar que cargó bien
```

Salida con el número de clases y las primeras 5:

```
Número de clases COCO: 80
Primeras 5 clases: ['person', 'bicycle', 'car', 'motorcycle', 'airplane']
```

```
def coco_ann_to_yolo_lines(coco_api, image_id, img_width, img_height):
    """Devuelve (lineas_deteccion, lineas_segmentacion) en formato YOLO para una imagen."""
    ann_ids = coco_api.getAnnIds(image_id, iscrowd=False) # ignora anotaciones "crowd" (grupos, no objetos individuales)
    anns = coco_api.loadAnns(ann_ids)

    det_lines, seg_lines = [], []
    for ann in anns: # recorre cada objeto anotado en la imagen
        x, y, w, h = ann["bbox"] # esquina superior-izq. + ancho/alto, en píxeles
        if w <= 0 or h <= 0: # descarta cajas inválidas/vacías
            continue
        cls_idx = catid_to_yoloidx[ann["category_id"]] # convierte el ID de COCO al índice contiguo de YOLO

        # Normaliza la caja a formato YOLO (centro + tamaño, 0-1)
        x_center_n = (x + w / 2) / img_width
        y_center_n = (y + h / 2) / img_height
        w_n = w / img_width
        h_n = h / img_height
        det_lines.append(f"{cls_idx} {x_center_n:.6f} {y_center_n:.6f} {w_n:.6f} {h_n:.6f}") # línea de detección lista

        segmentation = ann.get("segmentation")
        # Las anotaciones tipo RLE (objetos "crowd") no traen polígono utilizable; se omiten de seg.
        if isinstance(segmentation, list) and len(segmentation) > 0:
            polygon = max(segmentation, key=len) # si el objeto tiene varias partes separadas, toma la más grande
            if len(polygon) >= 6: # al menos 3 puntos (x, y) para formar un polígono válido
                coords = []
                for i in range(0, len(polygon) - 1, 2): # recorre pares (x, y) del polígono
                    px = min(max(polygon[i] / img_width, 0.0), 1.0) # normaliza y recorta a [0,1]
                    py = min(max(polygon[i + 1] / img_height, 0.0), 1.0)
                    coords.append(f"{px:.6f} {py:.6f}")
                seg_lines.append(f"{cls_idx} " + " ".join(coords)) # línea de segmentación lista

    return det_lines, seg_lines
```

Se generan dos "vistas" del dataset (det/ y seg/), escribiendo cada imagen y su etiqueta directamente en ambas, archivo por archivo (para que un rerun a medias siempre pueda completarse sin quedar con copias desactualizadas):



```
DET_VIEW = DATASET_ROOT / "det" # carpeta final que usará el modelo de detección (imágenes + labels de caja)
SEG_VIEW = DATASET_ROOT / "seg" # carpeta final que usará el modelo de segmentación (imágenes + labels de polígono)

def link_or_copy(src: Path, dst: Path):
    """Enlaza el archivo (symlink) o lo copia si el SO no lo permite. Revisa archivo por
    archivo, no la carpeta completa, para que un rerun a medias complete lo que faltó."""
    if dst.exists() or dst.is_symlink(): # si ya existe (o es un link ya creado), no repite trabajo
        return
    dst.parent.mkdir(parents=True, exist_ok=True)
    try:
        os.symlink(src, dst) # intenta enlazar en vez de copiar (ahorra espacio en disco)
    except OSError:
        shutil.copy2(src, dst) # si el symlink falla por permisos, copia el archivo real

def materialize_split(split_name, entries):
    """Escribe imágenes (symlink/copia) + labels de detección y de segmentación DIRECTAMENTE
    en las carpetas det/ y seg/ (sin una capa intermedia de symlink de carpeta completa, que
    en Windows sin modo desarrollador degenera en una copia estática que nunca se actualiza)."""
    det_images_out = DET_VIEW / "images" / split_name
    det_labels_out = DET_VIEW / "labels" / split_name
    seg_images_out = SEG_VIEW / "images" / split_name
    seg_labels_out = SEG_VIEW / "labels" / split_name
    for d in (det_images_out, det_labels_out, seg_images_out, seg_labels_out):
        d.mkdir(parents=True, exist_ok=True) # crea las 4 carpetas de este split si no existen

    for entry in entries: # una imagen a la vez
        src_img = COCO_RAW_ROOT / entry["subdir"] / entry["file_name"] # imagen original descargada
        link_or_copy(src_img, det_images_out / entry["file_name"]) # la deja disponible para detección
        link_or_copy(src_img, seg_images_out / entry["file_name"]) # y también para segmentación

        det_lines, seg_lines = coco_ann_to_yolo_lines( # genera las etiquetas de esta imagen
            entry["coco_api"], entry["image_id"], entry["width"], entry["height"]
        )
        stem = Path(entry["file_name"]).stem
        (det_labels_out / f"{stem}.txt").write_text("\n".join(det_lines)) # guarda etiqueta de detección
        (seg_labels_out / f"{stem}.txt").write_text("\n".join(seg_lines)) # guarda etiqueta de segmentación

    for split_name, entries in split_entries.items():
        print(f"Materializando split '{split_name}' ({len(entries)} imágenes)...")
        materialize_split(split_name, entries) # ejecuta todo el proceso para train, val y test

print("Anotaciones de detección y segmentación listas (en det/ y seg/).")
```

Salida de esta celda (materializando train/val/test):

```
Materializando split 'train' (1750 imágenes)...
Materializando split 'val' (375 imágenes)...
Materializando split 'test' (375 imágenes)...
Anotaciones de detección y segmentación listas (en det/ y seg/).
```

```
# Ya no hace falta nada aquí: la Sección 4 ya escribió las imágenes y etiquetas
# directamente en DET_VIEW y SEG_VIEW. Solo confirmamos que quedaron completas.
for view_name, view_root in [("det", DET_VIEW), ("seg", SEG_VIEW)]: # recorre las dos vistas del dataset
    for split_name in split_entries: # recorre train, val, test
        n_images = len(list((view_root / "images" / split_name).glob("*.jpg"))) # cuenta imágenes en esa carpeta
        n_labels = len(list((view_root / "labels" / split_name).glob("*.txt"))) # cuenta archivos de etiqueta
        print(f"({view_name}/{split_name}): {n_images} imágenes, {n_labels} archivos de etiquetas") # deben coincidir
```



Salida de la verificación (conteo de imágenes/etiquetas en det/ y seg/, por Split):

```
det/train: 1750 imágenes, 1750 archivos de etiquetas
det/val: 375 imágenes, 375 archivos de etiquetas
det/test: 375 imágenes, 375 archivos de etiquetas
seg/train: 1750 imágenes, 1750 archivos de etiquetas
seg/val: 375 imágenes, 375 archivos de etiquetas
seg/test: 375 imágenes, 375 archivos de etiquetas
```

Por último, se escribe el archivo .yaml que necesita Ultralytics para cada modelo:

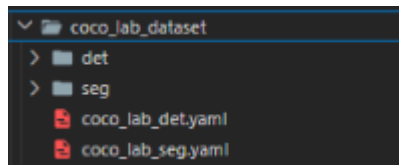
```
def write_dataset_yaml(path, dataset_view_root):
    config = {
        "path": str(dataset_view_root),      # carpeta raíz de esta vista (det/ o seg/)
        "train": "images/train",             # subcarpeta de entrenamiento, relativa a "path"
        "val": "images/val",                 # subcarpeta de validación
        "test": "images/test",               # subcarpeta de prueba
        "names": {idx: name for idx, name in enumerate(class_names)}, # mapa índice -> nombre de clase
    }
    with open(path, "w", encoding="utf-8") as f:
        yaml.safe_dump(config, f, sort_keys=False, allow_unicode=True) # escribe el archivo .yaml
    return path

det_yaml_path = write_dataset_yaml(DATASET_ROOT / "coco_lab_det.yaml", DET_VIEW) # yaml que usará el modelo de detección
seg_yaml_path = write_dataset_yaml(DATASET_ROOT / "coco_lab_seg.yaml", SEG_VIEW) # yaml que usará el modelo de segmentación

print("YAML de detección: ", det_yaml_path)
print("YAML de segmentación:", seg_yaml_path)
```

Rutas impresas de los dos archivos .yaml generados:

```
YAML de detección: E:\28\3, Taller Altamente Especializado IA - CINVESTAV\Módulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\coco_lab_det.yaml
YAML de segmentación: E:\28\3, Taller Altamente Especializado IA - CINVESTAV\Módulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\coco_lab_seg.yaml
```





1.5. 5 – 6. MODELO DE DETECCIÓN: CARGA Y ENTRENAMIENTO

Se carga yolo11n.pt (ya preentrenado en estas mismas 80 clases de COCO) y se afina sobre el split de entrenamiento. El entrenamiento es reanudable: si se interrumpe, al volver a correr esta celda continúa desde el último checkpoint en vez de reiniciar desde cero.

```
# 5. Cargar modelo de detección preentrenado.
# Si ya existe un entrenamiento previo interrumpido (last.pt), se reanuda desde ahí en
# vez de empezar de cero -- así puedes apagar la PC entre sesiones sin perder progreso.
det_lastckpt = Path("coco_lab_runs/detection/weights/last.pt") # checkpoint de una corrida anterior, si existe

if det_lastckpt.exists():
    print(f"Reanudando entrenamiento de detección desde: {det_lastckpt}")
    det_model = YOLO(str(det_lastckpt)) # carga el checkpoint a medias
    det_train_results = det_model.train(resume=True) # continúa desde donde se quedó
else:
    print("Iniciando entrenamiento de detección desde cero.")
    det_model = YOLO(DET_WEIGHTS) # carga los pesos preentrenados originales
    det_train_results = det_model.train( # 6. Entrenar con el split de entrenamiento
        data=str(det_yaml_path), # yaml del dataset de detección
        epochs=EPOCHS, # número de épocas (Sección 0)
        imgsz=IMG_SIZE, # tamaño de imagen de entrada
        batch=BATCH_SIZE, # tamaño de batch
        device=DEVICE, # CPU o GPU
        seed=SEED, # reproducibilidad
        project="coco_lab_runs", # carpeta donde se guardan los resultados
        name="detection", # subcarpeta específica de este modelo
        val=True, # valida contra el split "val" al final de cada época
        patience=PATIENCE, # corte anticipado si no mejora
        cache=CACHE_MODE, # cachea imágenes procesadas en disco
        workers=WORKERS, # hilos de carga de datos
    )
```

Consola mientras entrena detección (con las 5 épocas):

```
Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
99/100 0G 0.6266 0.5202 1.061 13 416: 100% 1/1 1.2s/it 1.2s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 2.0it/s 0.5s
all 4 17 0.883 0.291 0.406 0.25

Epoch GPU_mem box_loss cls_loss dfl_loss Instances Size
100/100 0G 0.522 0.5234 0.8506 13 416: 100% 1/1 1.5s/it 1.5s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 2.0it/s 0.5s
all 4 17 0.881 0.297 0.405 0.253

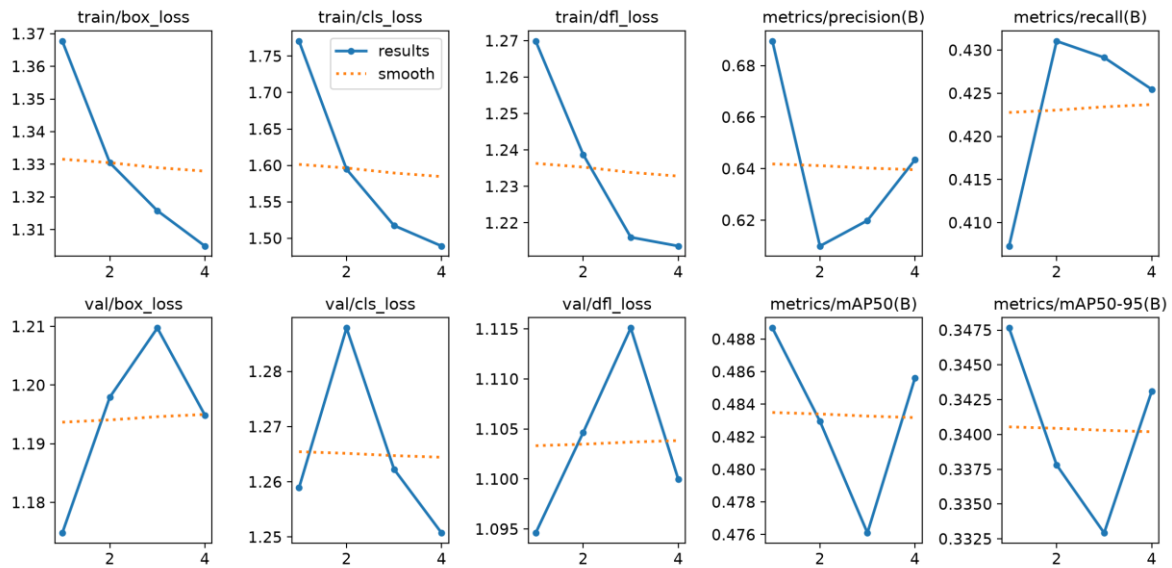
100 epochs completed in 0.079 hours.
Optimizer stripped from E:\28\3. Taller Altamente Especializado IA - CINESTAV\Mdulo 4\Laboratorios\Segmentation_Detection\runs\detect\train\weights\last.pt, 5.5MB
Optimizer stripped from E:\28\3. Taller Altamente Especializado IA - CINESTAV\Mdulo 4\Laboratorios\Segmentation_Detection\runs\detect\train\weights\best.pt, 5.5MB

Validating E:\28\3. Taller Altamente Especializado IA - CINESTAV\Mdulo 4\Laboratorios\Segmentation_Detection\runs\detect\train\weights\best.pt...
Ultralytics 8.4.106 Python-3.13.5 torch-2.13.0+cpu CPU (13th Gen Intel Core i3-1305U)
YOLO11n summary (fused): 101 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% 1/1 1.9it/s 0.5s
all 4 17 0.681 0.65 0.651 0.464
person 3 10 0.469 0.4 0.467 0.24
dog 1 1 0.688 1 0.995 0.796
horse 1 2 0.497 1 0.828 0.48
elephant 1 2 0.868 0.5 0.62 0.371
umbrella 1 1 0.564 1 0.995 0.895
potted plant 1 1 1 0 0 0

Speed: 1.6ms preprocess, 110.3ms inference, 0.0ms loss, 3.5ms postprocess per image
Results saved to E:\28\3. Taller Altamente Especializado IA - CINESTAV\Mdulo 4\Laboratorios\Segmentation_Detection\runs\detect\train
```



Imagen: coco_lab_runs/detection/results.png (curvas de entrenamiento generadas por Ultralytics):



Interpretación de la curva: las pérdidas de entrenamiento bajan de forma consistente en las 4 épocas, pero las de validación suben hasta la época 3 y solo bajan parcialmente en la 4, señal de sobreajuste leve. Esto se confirma en las métricas: mAP50(B) y mAP50-95(B) son más altas en la época 1 que en cualquier época posterior, así que seguir entrenando no mejoró la generalización del modelo, probablemente por lo chico del subconjunto (1,750 imágenes).



1.6. 7 – 8. MODELO DE SEGMENTACIÓN: CARGA Y ENTRENAMIENTO

Mismo mecanismo, ahora con yolo11n-seg.pt sobre las etiquetas de polígono.

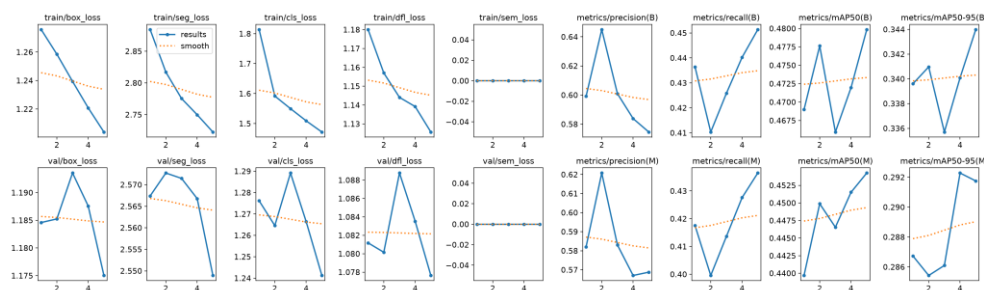
```
# 7. Cargar modelo de segmentación preentrenado (mismo mecanismo de reanudación que detección).
seg_last_ckpt = Path("coco_lab_runs/segmentation/weights/last.pt") # checkpoint de una corrida anterior, si existe

if seg_last_ckpt.exists():
    print(f"Reanudando entrenamiento de segmentación desde: {seg_last_ckpt}")
    seg_model = YOLO(str(seg_last_ckpt)) # carga el checkpoint a medias
    seg_train_results = seg_model.train(resume=True) # continúa desde donde se quedó
else:
    print("Iniciando entrenamiento de segmentación desde cero.")
    seg_model = YOLO(SEG_WEIGHTS) # carga los pesos preentrenados originales
    seg_train_results = seg_model.train( # 8. Entrenar con el split de entrenamiento
        data=str(seg_yaml_path), # yaml del dataset de segmentación
        epochs=EPOCHS, # número de épocas (Sección 0)
        imgsz=IMG_SIZE, # tamaño de imagen de entrada
        batch=BATCH_SIZE, # tamaño de batch
        device=DEVICE, # CPU o GPU
        seed=SEED, # reproducibilidad
        project="coco_lab_runs", # carpeta donde se guardan los resultados
        name="segmentation", # subcarpeta específica de este modelo
        val=True, # valida contra el split "val" al final de cada época
        patience=PATIENCE, # corte anticipado si no mejora
        cache=CACHE_MODE, # cachea imágenes procesadas en disco
        workers=WORKERS, # hilos de carga de datos
    )
```

Imagen: consola mientras entrena segmentación (con las 5 épocas):

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	416: 100%	110/110 6.4s/it 11:495.2ss
1/5	0G	1.276	2.884	1.814	1.18	0	91	Mask(P)	R	mAP50 mAP50-95: 100%
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	0.582	0.418	0.44 0.287
	all	375	2654	0.599	0.436	0.469	0.34			
Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	416: 100%	110/110 7.0s/it 12:467.3ss
2/5	0G	1.259	2.817	1.592	1.157	0	82	Mask(P)	R	mAP50 mAP50-95: 100%
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	0.621	0.4	0.45 0.285
	all	375	2654	0.645	0.41	0.478	0.341			
Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	416: 100%	110/110 8.0s/it 14:434.4ss
3/5	0G	1.239	2.775	1.55	1.144	0	85	Mask(P)	R	mAP50 mAP50-95: 100%
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	0.583	0.414	0.447 0.286
	all	375	2654	0.601	0.426	0.466	0.336			
Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	416: 100%	110/110 5.5s/it 10:084.1ss
4/5	0G	1.221	2.75	1.509	1.139	0	98	Mask(P)	R	mAP50 mAP50-95: 100%
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	0.567	0.428	0.452 0.292
	all	375	2654	0.584	0.44	0.472	0.34			
Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	sem_loss	Instances	Size	416: 100%	110/110 5.1s/it 9:243.7ss
5/5	0G	1.204	2.723	1.472	1.126	0	98	Mask(P)	R	mAP50 mAP50-95: 100%
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	0.569	0.436	0.454 0.292
	all	375	2654	0.575	0.451	0.48	0.344			

Imagen: coco_lab_runs/segmentation/results.png (curvas de entrenamiento):





1.7. EVALUACIÓN DE AMBOS MODELOS EN EL CONJUNTO DE TEST

Ambos modelos se evalúan contra el split de test, nunca visto durante el entrenamiento:

```
det_test_metrics = det_model.val(
    data=str(det_yaml_path), # mismo yaml de detección usado en entrenamiento
    split="test",           # evalúa contra el split de prueba, nunca visto antes
    imgsz=IMG_SIZE,         # mismo tamaño de imagen que en entrenamiento
    device=DEVICE,          # CPU o GPU
)
```

Imagen: salida de det_model.val(..., split='test')

```
val: Scanning E:\2023_Taller Altimante Especializado IA - CINESTAV\Modulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\det\labels\test... 375 Images, 6 backgrounds, 0 corrupt: 100% 375/375 66.71it/s 5.6s.
val: New cache created: E:\2023_Taller Altimante Especializado IA - CINESTAV\Modulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\det\labels\test.cache
Class Images Instances Box(P R mAP50 mAP50-95) 100% 24/24 1.0it/s 23.3s1.8ss
all 375 2623 0.570 0.474 0.506 0.258
person 194 805 0.66 0.591 0.64 0.424
bicycle 10 19 0.863 0.368 0.375 0.227
car 36 126 0.44 0.389 0.381 0.23
motorcycle 13 25 0.57 0.52 0.633 0.428
airplane 5 6 0.855 0.588 0.940 0.796
bus 12 16 0.447 0.625 0.65 0.571
train 14 15 0.92 0.771 0.894 0.732
truck 22 32 0.541 0.375 0.438 0.278
boat 9 23 0.514 0.478 0.504 0.24
traffic light 9 27 0.638 0.444 0.492 0.271
fire hydrant 6 7 0.792 0.857 0.815 0.683
stop sign 3 3 0.843 1 0.995 0.962
parking meter 2 3 0.989 0.333 0.395 0.088
bench 17 24 0.425 0.125 0.160 0.155
bird 10 39 0.352 0.41 0.447 0.289
cat 15 17 0.879 0.854 0.927 0.854
dog 14 19 0.612 0.789 0.776 0.63
horse 9 17 0.583 0.529 0.588 0.455
sheep 3 15 0.339 0.6 0.487 0.254
cow 8 23 0.715 0.826 0.840 0.657
elephant 4 22 0.76 0.576 0.714 0.409
bear 7 13 1 0.533 0.952 0.764
zebra 7 30 0.728 0.7 0.823 0.579
giraffe 15 20 0.943 0.9 0.964 0.871
```

```
seg_test_metrics = seg_model.val(
    data=str(seg_yaml_path), # mismo yaml de segmentación usado en entrenamiento
    split="test",           # evalúa contra el split de prueba, nunca visto antes
    imgsz=IMG_SIZE,         # mismo tamaño de imagen que en entrenamiento
    device=DEVICE,          # CPU o GPU
)
```

Imagen: salida de seg_model.val(..., split='test')

```
Ultralytics 8.4.106 Python-3.13.5 torch-2.13.0+cpu CPU (13th Gen Intel Core i3-13050)
YOLO11n-seg summary (Fused): 114 layers, 2,868,664 parameters, 0 gradients, 9.7 GFLOPs
WARNING val: Slow image access detected (ping: 0.18.0 ms, read: 23.222.3 MB/s, size: 130.7 KB). Use local storage instead of remote/mounted storage for better performance. See https://docs.ultralytics.com/guides/model-training-tips/
val: Scanning E:\2023_Taller Altimante Especializado IA - CINESTAV\Modulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\seg\labels\test... 375 Images, 6 backgrounds, 0 corrupt: 100% 375/375 44.71it/s 8.4s0.1s
val: New cache created: E:\2023_Taller Altimante Especializado IA - CINESTAV\Modulo 4\Laboratorios\Segmentation_Detection\coco_lab_dataset\seg\labels\test.cache
Class Images Instances Box(P R mAP50 mAP50-95) Mask(P R mAP50 mAP50-95) 100% 24/24 1.4s/it 33.8s1.4ss
all 375 2623 0.611 0.439 0.494 0.345 0.635 0.418 0.477 0.3
person 194 805 0.888 0.547 0.635 0.425 0.809 0.519 0.686 0.336
bicycle 10 19 0.738 0.208 0.291 0.202 0.72 0.263 0.287 0.152
car 36 126 0.585 0.346 0.371 0.232 0.659 0.322 0.349 0.187
motorcycle 13 25 0.822 0.48 0.588 0.388 0.85 0.454 0.645 0.362
airplane 5 6 0.669 0.833 0.888 0.812 0.838 1 0.972 0.587
bus 12 16 0.669 0.6 0.661 0.523 0.703 0.5 0.548 0.436
train 14 15 0.759 0.667 0.71 0.681 0.77 0.667 0.71 0.593
truck 22 32 0.763 0.188 0.514 0.315 0.802 0.188 0.449 0.245
boat 9 23 0.669 0.615 0.709 0.328 0.766 0.57 0.65 0.268
traffic light 9 27 0.614 0.444 0.466 0.277 0.503 0.407 0.461 0.235
fire hydrant 6 7 0.756 0.857 0.855 0.681 0.779 0.857 0.855 0.545
stop sign 3 3 0.73 0.667 0.913 0.863 0.764 0.667 0.913 0.764
parking meter 2 3 0.586 0.667 0.443 0.128 0.307 0.333 0.112 0.0447
bench 17 24 0.323 0.0833 0.174 0.14 0.403 0.0833 0.126 0.086
bird 10 39 0.459 0.308 0.362 0.214 0.606 0.308 0.332 0.178
cat 15 17 0.706 0.941 0.986 0.814 0.971 0.866 0.906 0.72
dog 14 19 0.72 0.842 0.882 0.76 0.888 0.838 0.882 0.684
horse 9 17 0.669 0.412 0.521 0.333 0.72 0.412 0.439 0.265
...
teddy bear 5 10 1 0.787 0.853 0.662 1 0.778 0.857 0.592
toothbrush 1 1 0.657 1 0.995 0.997 0.783 1 0.995 0.697
Speed: 0.6ms preprocess, 54.8ms inference, 0.0ms loss, 6.1ms postprocess per image
```



1.8. MÉTRICAS DE DETECCIÓN (CONJUNTO DE TEST)

```
det_precision = float(det_test_metrics.box.mp) # precisión media sobre las 80 clases
det_recall = float(det_test_metrics.box.mr) # recall medio sobre las 80 clases
det_f1 = 2 * det_precision * det_recall / (det_precision + det_recall + 1e-9) # F1 derivado (evita división entre 0)
det_map50 = float(det_test_metrics.box.map50) # mAP con umbral IoU=0.50 (más permisivo)
det_map5095 = float(det_test_metrics.box.map) # mAP promedio sobre IoU 0.50 a 0.95 (más estricto)

print("Métricas de detección - conjunto de test")
print(f" Precisión (media sobre clases): {det_precision:.4f}")
print(f" Recall (media sobre clases): {det_recall:.4f}")
print(f" F1-score: {det_f1:.4f}")
print(f" mAP@0.50: {det_map50:.4f}")
print(f" mAP@0.50:0.95: {det_map5095:.4f}")
```

Imagen: salida con los 5 valores de detección impresos

```
Métricas de detección - conjunto de test
Precisión (media sobre clases): 0.5794
Recall (media sobre clases): 0.4741
F1-score: 0.5214
mAP@0.50: 0.5059
mAP@0.50:0.95: 0.3576
```

Resultados obtenidos sobre el conjunto de test para el modelo de detección:

Métrica	Definición	Valor	Cómo interpretarlo
Precisión	VP / (VP + FP), promedio sobre las 80 clases	0.5794	Qué proporción de las cajas predichas eran correctas.
Recall	VP / (VP + FN), promedio sobre las 80 clases	0.4741	Qué proporción de los objetos reales fueron detectados.
F1-score	2 · Precisión · Recall / (Precisión + Recall)	0.5214	Balance entre precisión y recall en un solo número.
mAP@0.50	AP promedio con umbral IoU=0.50	0.5059	Métrica permisiva, estándar más citado en detección.
mAP@0.50:0.95	AP promedio sobre 10 umbrales IoU (0.50-0.95)	0.3576	Métrica estricta, la principal del benchmark oficial de COCO.



1.9. MÉTRICAS DE SEGMENTACIÓN (CONJUNTO DE TEST)

```
seg_mask_precision = float(seg_test_metrics.seg.mp) # precisión de máscara, media sobre clases
seg_mask_recall = float(seg_test_metrics.seg.mr) # recall de máscara, media sobre clases
seg_mask_map50 = float(seg_test_metrics.seg.map50) # mAP de máscara con IoU=0.50
seg_mask_map5095 = float(seg_test_metrics.seg.map) # mAP de máscara promedio sobre IoU 0.50 a 0.95

print("Métricas de segmentación - conjunto de test (Ultralytics)")
print(f" Precisión de máscara (media): {seg_mask_precision:.4f}")
print(f" Recall de máscara (media): {seg_mask_recall:.4f}")
print(f" Mask mAP@0.50: {seg_mask_map50:.4f}")
print(f" Mask mAP@0.50:0.95: {seg_mask_map5095:.4f}")
```

Imagen: salida con los 4 valores de segmentación impresos (Ultralytics)

```
Métricas de segmentación - conjunto de test (Ultralytics)
Precisión de máscara (media): 0.6352
Recall de máscara (media): 0.4183
Mask mAP@0.50: 0.4766
Mask mAP@0.50:0.95: 0.2998
```

Adicionalmente, se calculan IoU y Dice explícitos a nivel de píxel, sobre el conjunto de test completo, imagen: salida con los valores de IoU y Dice a nivel de píxel.

```
IoU a nivel de píxel (máscara combinada, 375 imágenes de test): 0.7103
Dice a nivel de píxel (mismas 375 imágenes): 0.8306

Nota: Dice = 2*IoU / (1+IoU) matemáticamente, así que ambos números son consistentes entre sí.
```

1.10. VISUALIZACIÓN DE RESULTADOS EN IMÁGENES DE TEST





Cinvestav
Guadalajara



COCYTEN
CONSEJO DE CIENCIA Y TECNOLOGÍA
DEL ESTADO DE NAYARIT



Nayarit
NUESTRA DIGNIDAD Y COMPROMISO



CONCLUSION

El modelo de detección obtuvo, sobre el conjunto de test, una precisión de 0.579, recall de 0.474, F1 de 0.521, mAP@0.50 de 0.506 y mAP@0.50:0.95 de 0.358. El modelo de segmentación logró una precisión de máscara de 0.635, recall de máscara de 0.418, mask mAP@0.50 de 0.477 y mask mAP@0.50:0.95 de 0.300, con un IoU a nivel de píxel de 0.710 y un Dice de 0.831, este último notablemente alto, lo que indica buen traslape entre las máscaras predichas y las reales pese a las pocas épocas de entrenamiento.

Las curvas de entrenamiento (Sección 5-6) ya habían anticipado esto: las métricas de validación fueron mejores en la primera época que, en las siguientes, señal de que el fine-tuning sobre un subconjunto tan chico (1,750 imágenes de entrenamiento) tiende a sobreajustar rápido en vez de seguir generalizando. Aun así, ambos modelos parten de pesos ya preentrenados en estas mismas 80 clases, lo que explica por qué los resultados son razonables desde pocas épocas.

Para mejorar estos resultados, valdría la pena: usar un subconjunto más grande de imágenes, aplicar más aumentos de datos (data augmentation) para compensar el tamaño chico del dataset, y detener el entrenamiento en la época con mejor métrica de validación (early stopping real) en vez de usar siempre la última época.